

Yan Li
Final Project

Project Overview

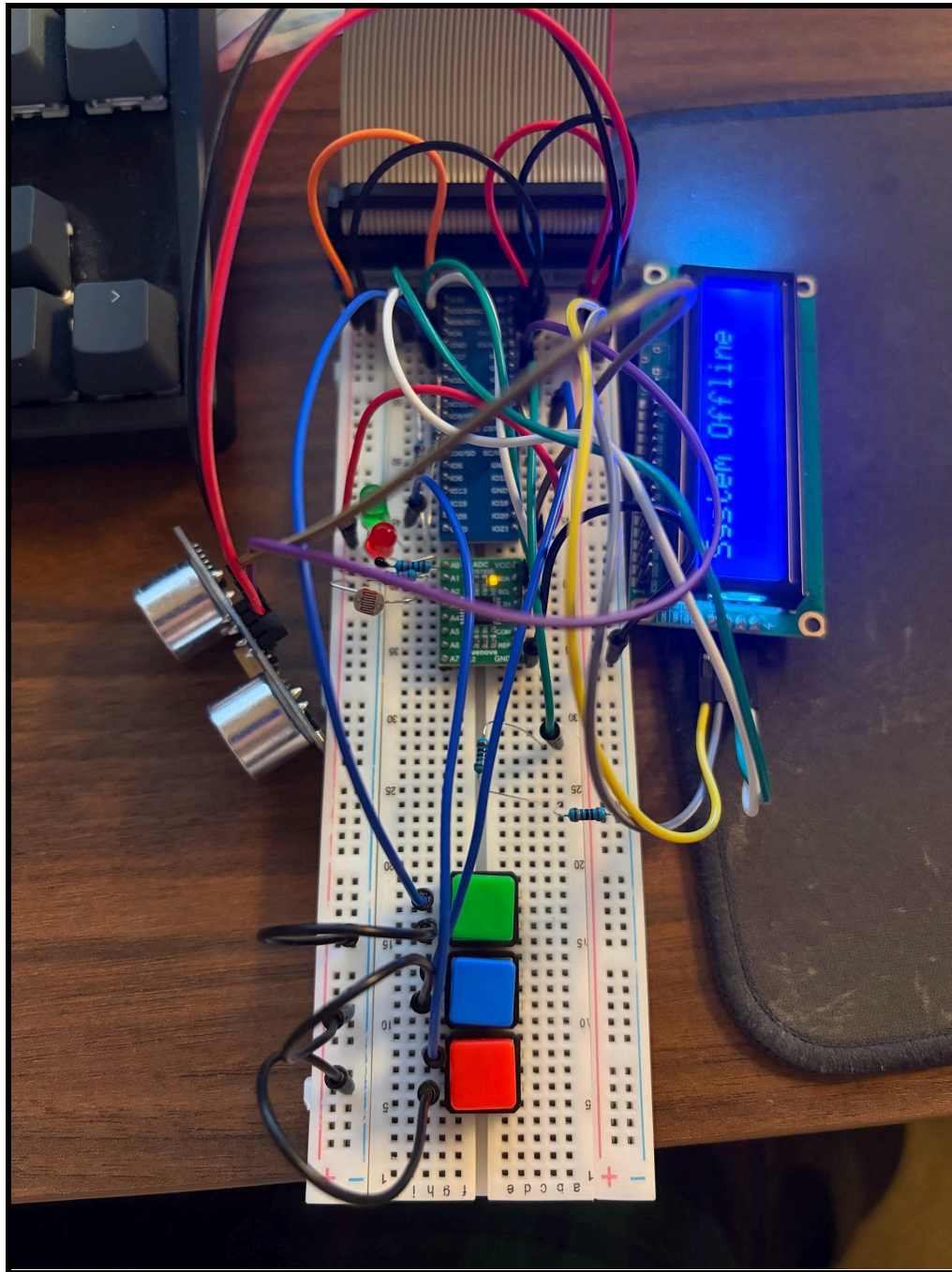
For this project, I implemented a home security system in python with a temperature sensor, ultrasonic sensor, and LDR. Alerts and notifications are sent to a LCD. An email alert system is also in place using postfix.

I also implemented a menu system for extra credit. Three buttons are used to navigate the menus.

Hardware Components

- Breadboard
- Wires
- Green and Red LED
- Various Resistors
- ADC Module
- LCD Module
- GPIO Extension Board
- Thermistor
- Photoresistor
- Ultrasonic Sensor
- 3 Buttons
- Raspberry Pi 5

Circuit Wiring



Circuit Wiring Explanations

ADC

In this project, I used the GPIO extension board. Starting with the ADC wiring, it is powered to the 3.3v rail and also grounded. SDA is connected to GPIO2, and SCL is connected to GPIO3. Also on the ADC, the thermistor is connected to A0 with a 10k resistor to 3.3v. The LDR is also connected to a 10k resistor to the 3.3v rail.

Buttons

There are three buttons; green, blue, and red. For the purposes of this project, the green is used to enter normal mode, the red used to enter security mode, and blue used to access the menu to change modes.

All buttons are grounded. The green button is connected to GPIO17. The blue button is connected to GPIO25. The red button is connected to GPIO 26.

LCD

The LCD is connected to the 5v power rail and ground. SDA is connected to GPIO2, and SCL is connected to GPIO3. These are shared with the ADC.

Ultrasonic Sensor

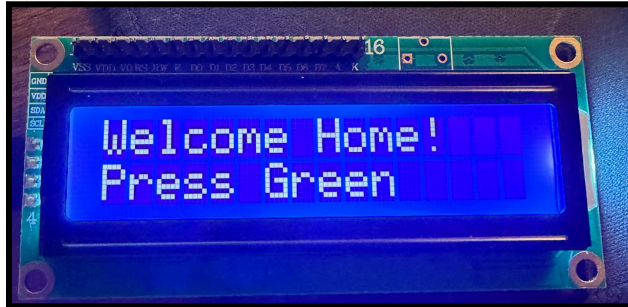
This sensor required a little more work. VCC is connected to 5v, GND is connected to ground, and Trip is connected to GPIO23. Echo is connected using a voltage divider. From the sensor, echo is connected to a 1k resistor, then to GPIO 24. Also from GPIO24 are two 1k resistors in series that connect to ground.

LED

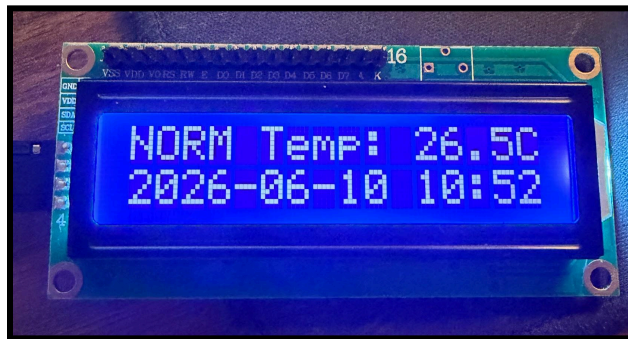
The green LED is connected to GPIO 27 with a 220ohm resistor. The red LED is connected to GPIO 22 with a 220ohm resistor.

Normal Mode Operation

When the user first starts the program, they are greeted with a simple welcome menu. It asks the user to press the green button, which then takes them to normal mode.



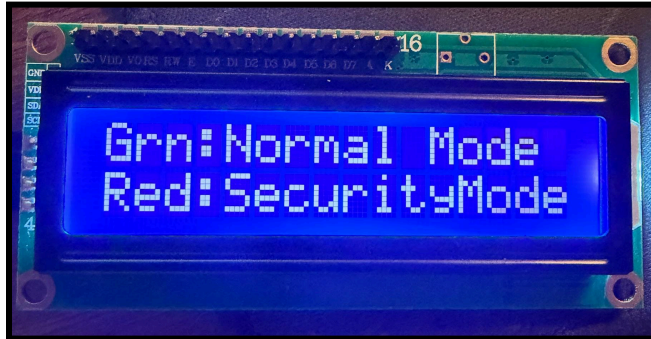
After pressing green, they enter normal mode,



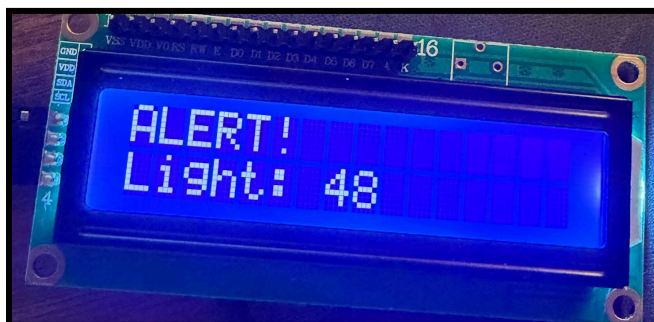
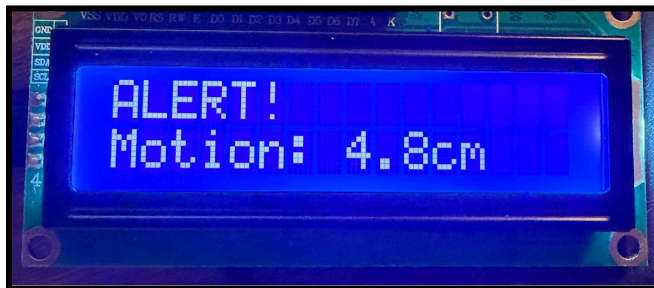
In normal mode, the LCD will display the current temperature, date, and time.

Security Mode Operation

To enter security mode, the user must press the blue button to enter the menu. From there, they can choose to enter either normal mode or security mode. Press red to enter security mode from the menu,



While in security mode, the LCD will display alerts depending on the type of alert (motion detected or change in light),



Sensors Explanation

The thermistor is used to read the temperature and is then converted to celsius in the code. It is constantly updated.

```
def get_temperature():  
    raw_val = read_adc(0) # A0  
    celsius = (raw_val / 255.0) * 50.0  
    return round(celsius, 1)
```

On the LCD, this information is updated every second. You can also see the function used to get the date and time.

```
# Update time to LCD  
now_str = datetime.datetime.now().strftime("%Y-%m-%d %I:%M %p")  
# Update temp to LCD  
temp = get_temperature()  
update_lcd_display(f"NORM Temp: {temp}C", now_str)  
time.sleep(1.0) # Delay between updates
```

The LDR is used when in security mode. It will trigger an alert if there is a change in light levels past a certain threshold. When we enter security mode, the program saves the value of the current light level at the time of mode switch. The LDR has a range of 0-255.

```
DISTANCE_THRESHOLD_CM = 30.0  
LIGHT_CHANGE_THRESHOLD = 40
```

In security mode, the program constantly reads new distance values from the ultrasonic sensor and light values from the LDR. If it detects something within 30cm, or a change in light levels greater than 40, it triggers an alert.

Notification Method

To send email notifications, I used postfix. I used the following function to do so,

```
def send_security_alert(event_type, distance, light_val, current_time):  
    global email_cooldown_timer  
    # Emails every 30 seconds to prevent spam  
    if time.time() < email_cooldown_timer:  
        return  
  
    print(f"[Action] Dispatching Alert Email for: {event_type}...")  
    subject = "Raspberry Pi Security Alert"  
    body = f"""Security Alert!  
Mode: Security Mode  
Event: {event_type}  
Ultrasonic distance: {distance} cm  
Light sensor value: {light_val}  
Time: {current_time}"""  
  
    msg = MIMEText(body)  
    msg['Subject'] = subject  
    msg['From'] = "smart-home@raspberrypi.local"  
    msg['To'] = email_recipient  
  
    server = smtplib.SMTP('localhost', 25)  
    server.sendmail(msg['From'], [msg['To']], msg.as_string())  
    server.quit()  
    print("[Success] Email alert delivered successfully.")  
    email_cooldown_timer = time.time() + 30
```

Built into this function is a timer. There is a 30 second cooldown between each email alert sent so that I am not spammed with emails.



In the email, all information at the time of the alert is shared.

Code Structure and Important Functions

Below is the main code. It is all in a while loop so that it keeps running until I tell the program to stop in the terminal. In the while loop, the program is able to detect change in modes and responds correspondingly.

```
def main():
    global state_changed
    # Setup Pins
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(BUTTON_GREEN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(BUTTON_BLUE, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(BUTTON_RED, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(LED_NORMAL, GPIO.OUT)
    GPIO.setup(LED_ALARM, GPIO.OUT)
    GPIO.setup(TRIG_PIN, GPIO.OUT)
    GPIO.setup(ECHO_PIN, GPIO.IN)

    GPIO.output(TRIG_PIN, False)
    lcd_init()

    print("System Activated") # Print to terminal
    GPIO.add_event_detect(BUTTON_GREEN, GPIO.FALLING, callback=button_pressed_callback, bouncetime=200)
    GPIO.add_event_detect(BUTTON_BLUE, GPIO.FALLING, callback=button_pressed_callback, bouncetime=200)
    GPIO.add_event_detect(BUTTON_RED, GPIO.FALLING, callback=button_pressed_callback, bouncetime=200)

    try:
        while True:
            if current_state == STATE_STARTUP:
                if state_changed:
                    GPIO.output(LED_NORMAL, GPIO.LOW)
                    GPIO.output(LED_ALARM, GPIO.LOW)
                    update_lcd_display("Welcome Home!", "Press Green")
                    state_changed = False
                    time.sleep(0.1)

            elif current_state == STATE_NORMAL:
                run_normal_mode()

            elif current_state == STATE_MENU:
                if state_changed:
                    GPIO.output(LED_NORMAL, GPIO.LOW)
                    update_lcd_display("Grn:Normal Mode", "Red:SecurityMode")
                    state_changed = False
                    time.sleep(0.1)

            elif current_state == STATE_SECURITY:
                run_security_mode()
    except KeyboardInterrupt:
        print("\n[System] Shutting Down")
    finally:
        update_lcd_display("System Offline", "")
        GPIO.cleanup()
```

The while loop is inside of the try. The point of this is so that I can terminate the program in the terminal by using ctrl + c. It will also display a shutdown message on the LCD.

```
[ALERT] Motion detected at 2026-06-10 10:52 PM!
Distance: 4.8 cm | Light ADC Value: 110 (Baseline: 113)
[Action] Dispatching Alert Email for: Motion detected...
[Success] Email alert delivered successfully.
```

Here is an example of an error message being displayed in the terminal, also confirming that an email notification was sent.

Normal Mode Function

```
def run_normal_mode():
    GPIO.output(LED_NORMAL, GPIO.HIGH) # Green LED on
    GPIO.output(LED_ALARM, GPIO.LOW)

    # Update time to LCD
    now_str = datetime.datetime.now().strftime("%Y-%m-%d %I:%M %p")
    # Update temp to LCD
    temp = get_temperature()
    update_lcd_display(f"NORM Temp: {temp}C", now_str)
    time.sleep(1.0) # Delay between updates
```

While running in normal mode, the program uses this function. It turns the green LED on to indicate that it is in normal mode. It also continuously updates the LCD with the current date, time, and temperature.

Security Mode Function

```
def run_security_mode():
    GPIO.output(LED_NORMAL, GPIO.LOW) # Green LED off
    distance = get_ultrasonic_distance()
    light_level = get_light_level()
    now_str = datetime.datetime.now().strftime("%Y-%m-%d %I:%M %p")
    update_lcd_display("SECURITY MODE", "Monitoring")

    # Threshold Parameters
    motion_breached = distance < DISTANCE_THRESHOLD_CM
    light_breached = abs(light_level - security_baseline_light) > LIGHT_CHANGE_THRESHOLD

    if motion_breached or light_breached:
        if motion_breached:
            event_msg = "Motion detected"
            update_lcd_display("ALERT!", f"Motion: {distance}cm")
        else: # Light breached
            event_msg = "Light anomaly"
            update_lcd_display("ALERT!", f"Light: {light_level}")

        print(f"\n[ALERT] {event_msg} at {now_str}!")
        print(f"Distance: {distance} cm | Light ADC Value: {light_level} (Baseline: {security_baseline_light})")

        # Send email and blink red LED
        send_security_alert(event_msg, distance, light_level, now_str)
        for x in range(3):
            GPIO.output(LED_ALARM, GPIO.HIGH)
            time.sleep(0.5)
            GPIO.output(LED_ALARM, GPIO.LOW)
            time.sleep(0.5)
    else:
        GPIO.output(LED_ALARM, GPIO.LOW)

    time.sleep(0.2)
```

This is the security mode function. First, it turns the green LED off to indicate that it is in security mode. It will save the ultrasonic distance, current light levels, and current date/time. It also calculates certain thresholds that were previously determined using global variables. If it detects that these thresholds are breached, it will send a notification to both the terminal and LCD. The red LED will blink to show an alert is detected. It will tell the user either motion was detected or light was changed. If an email notification has not been sent within the last 30 seconds, it will send one to the user with all of the information.

Mode Management Function

```
def button_pressed_callback(channel):
    global current_state, state_changed, security_baseline_light
    time.sleep(0.05)
    if GPIO.input(channel) == GPIO.HIGH:
        return

    # Startup Menu (press green to continue)
    if current_state == STATE_STARTUP:
        if channel == BUTTON_GREEN:
            current_state = STATE_NORMAL
            state_changed = True

    elif current_state == STATE_NORMAL:
        if channel == BUTTON_BLUE: # Press blue to enter menu
            current_state = STATE_MENU
            state_changed = True

    elif current_state == STATE_MENU:
        if channel == BUTTON_GREEN:
            current_state = STATE_NORMAL
            state_changed = True
        elif channel == BUTTON_RED:
            current_state = STATE_SECURITY
            security_baseline_light = get_light_level()
            state_changed = True

    elif current_state == STATE_SECURITY:
        if channel == BUTTON_BLUE: # Press blue to enter menu
            current_state = STATE_MENU
            state_changed = True
```

This function is used to control the three buttons and their actions. When the program first starts, we begin in `STATE_STARTUP`. This will display a welcome message and ask the user to press the green button to continue. It then enters `STATE_NORMAL`.

In `STATE_NORMAL`, we are in normal mode. From here, the user can press the blue button to enter the menu. The menu is accessed in `STATE_MENU`. From here, the user can choose either to enter normal mode or security mode. The green button enters normal mode (`STATE_NORMAL`) and the red button enters security mode (`STATE_SECURITY`).

Testing Results

The project was a complete success. I was able to implement every required functionality, and even added extra credit option 6, the improved user interface, by adding a menu system using buttons and the LCD display.

The system works as it should, entering both normal mode and security mode properly. Each sensor also works properly, displaying correct information when it should.

The email notification system was implemented using postfix and works properly.

The circuit and program was tested on a Raspberry Pi 5 on the Raspberry Pi OS. I used SSH to run the Pi headless from my Windows 11 laptop. All programming was done in Python, using nano in the Windows terminal.